■ CSAO Rail

Cart Super Add-On Recommendation System

Zomathon Hackathon — Problem Statement 2
Submitted by: Team Name
Date: March 2025

119,517	34	0.7931	■449	21%
Training Rows	Features	AUC-ROC	Avg AOV Lift	Accept Rate

1. Problem Understanding

The Cart Super Add-On (CSAO) Rail is a feature on Zomato's ordering interface that recommends complementary items to customers as they build their cart. The core challenge is building an intelligent, real-time recommendation system that understands the current cart composition and dynamically suggests the most relevant add-on items.

1.1 Business Challenge

- Low add-on acceptance rates due to generic, context-unaware recommendations
- Cart context changes dynamically — every item added changes what should be recommended next
- Cold start problem — new users have no history to learn from
- Strict latency constraint of 200–300ms at millions of requests per day
- Recommendations must work across diverse user segments, cities, and meal types

1.2 Problem Framing

> **We treat this as a Learning-to-Rank problem.** Given a cart state and a set of candidate items, we rank candidates by their probability of being added. This is superior to binary classification because it naturally handles the "top-K" nature of recommendations and optimizes for ranking quality (NDCG) rather than just accuracy.

1.3 Key Insight

The most powerful signal is **meal completeness** — most food orders follow predictable composition patterns. A cart with Biryani is "incomplete" until it has Raita, a beverage, and a dessert. By explicitly modeling these patterns (through co-occurrence data AND LLM reasoning), we can recommend items that feel natural and necessary rather than random.

2. Data Generation & Feature Engineering

2.1 Synthetic Data Design

Since real Zomato data is proprietary, we generated realistic synthetic data that mirrors actual food delivery behavior patterns across Indian cities.

Dataset	Rows	Columns	Key Purpose
cart_sessions.csv	119,517	30	Core training data — one row per recommendation moment
orders.csv	15,000	8	Historical orders with temporal context
order_items.csv	34,641	7	Item-level order details for co-occurrence
menu_items.csv	6,995	9	Restaurant menu with categories & prices
restaurants.csv	500	12	Restaurant profiles across 7 cities
users.csv	2,000	10	User profiles with segments & preferences

2.2 Realism Injected

- **Peak hour patterns:** Lunch (12–2pm) and dinner (7–10pm) have weighted higher order volumes
- **City-wise veg preference:** Chennai/Bangalore skew 60% vegetarian vs 35% for other cities
- **Cold start users:** 15% of users have sparse order history to simulate real-world new users
- **Meal completion logic:** Cart sessions built sequentially — Biryani → Raita → Dessert → Drink
- **Label imbalance:** 13.5% positive (accepted) vs 86.5% negative — mirrors real acceptance rates

2.3 Feature Engineering (34 Features)

Feature Group	Key Features	Business Rationale
Cart State (11)	cart_size, meal_completeness_score, has_beverage, has_dessert	Captures what's in cart and what's missing
Candidate Item (6)	cand_price, cand_category, popularity_score, cand_is_veg	Describes the item being considered
Cart-Item Relation (8)	cooccur_log, is_complementary_category, veg_match, price_ratio, affordability	Relationship between cart and candidate
User (4)	user_avg_spend, segment_encoded, user_is_veg, is_tier1_city	Who is ordering
Context (5)	meal_time_encoded, is_weekend, is_dinner_or_late, weekend_premium	When and where they're ordering

> **Key Engineering Insight:** The **cooccur_log** feature (log-transformed co-occurrence score) is the single strongest predictor with r = +0.30 correlation with the label. Items that are historically ordered together are much more likely to be accepted as add-on recommendations.

3. Model Architecture

3.1 Two-Stage Architecture

We adopt the industry-standard two-stage recommendation architecture used by companies like Google, Netflix, and Uber Eats — optimized for both speed and accuracy.

- **Stage 1 — Retrieval (fast, broad):** Co-occurrence filtering shortlists ~50 candidates from thousands in < 20ms. Uses historical item pairs to find plausible add-ons.
- **Stage 2 — Ranking (smart, precise):** LightGBM LambdaRank model scores and ranks the 50 candidates using all 34 features, returning top 8–10 in < 50ms.
- **LLM Layer (AI Edge):** Google Gemini analyzes meal completeness and re-ranks results to prioritize items filling missing meal components.

3.2 Why LightGBM LambdaRank?

- **Built for ranking:** LambdaRank directly optimizes NDCG — the right metric for our use case
- **Handles imbalance:** scale_pos_weight=6.4 compensates for 13.5% positive rate
- **Fast inference:** < 50ms per request even with 34 features
- **Interpretable:** Feature importance scores help explain recommendations to business stakeholders
- **Production ready:** Widely deployed at scale (used at Microsoft, Alibaba, Baidu)

3.3 Sequential Cart Modeling

Each recommendation is made with full awareness of what's already in the cart (step-by-step). The **step_ratio** and **meal_completeness_score** features capture how far along in the meal-building process the user is, enabling recommendations that evolve naturally as items are added.

3.4 Cold Start Strategy

Scenario	Strategy	Data Source
New user, no history	LLM infers preferences from city + time + segment	Gemini API
New restaurant	Popularity-based fallback using item categories	menu_items.csv
New menu item	Content-based similarity to existing items	item features
Sparse user (<3 orders)	Hybrid: 70% population model + 30% sparse history	users.csv

4. LLM Integration (AI Edge)

We integrate Google Gemini as a semantic reasoning layer on top of the ML model. This is our key differentiator — combining statistical learning with language understanding.

4.1 Use 1 — Meal Completeness Analyzer

For every cart state, Gemini analyzes the cart items and identifies missing meal components. This response drives the **llm_category_boost** feature that re-ranks candidates to prioritize items filling meal gaps.

Cart Items	Meal Time	LLM Analysis	Score
Chicken Biryani	Dinner	Missing: raita, beverage, dessert	4/10
Butter Chicken + Naan	Dinner	Missing: beverage, dessert	5/10
Masala Dosa	Breakfast	Missing: filter coffee/beverage	6/10
Pizza + Garlic Bread	Lunch	Missing: beverage, dessert	4/10
Paneer Tikka + Dal + Naan	Dinner	Missing: beverage, dessert	5/10

4.2 Use 2 — Cold Start Profile Generator

For new users with no order history, Gemini generates a preference profile from contextual signals: city, vegetarian preference, time of day, and user segment.

User Context	LLM Inferred Profile	Confidence
Chennai | Veg | Breakfast | Budget	South Indian comfort food | Price-sensitive	7/10
Delhi | Non-veg | Dinner | Premium	Continental multi-course | Low price sensitivity	8/10
Mumbai | Non-veg | Lunch | Mid-range	North Indian / Chinese | Medium sensitivity	6/10

4.3 LLM-Derived Features Added to Model

Feature	Description	Impact
llm_category_boost	Probability LLM recommends this category	Re-ranks candidates based on meal gaps
llm_completeness	LLM's meal completeness score (0–1)	Higher boost when meal is incomplete
llm_preference_confidence	Confidence in user preference inference	Weighs LLM signal in cold start

5. Model Evaluation & Results

5.1 Evaluation Methodology

- **Temporal split:** 80% train (earlier sessions) / 20% test (later sessions) — prevents data leakage
- **Class imbalance:** Handled via scale_pos_weight=6.4, weighted sampling
- **Metrics:** AUC-ROC (discrimination), Precision@K, Recall@K, NDCG@K (ranking quality)
- **Baseline:** Pure co-occurrence ranking — what the system would do without ML

5.2 Model Performance vs Baseline

Metric	Baseline (Co-occurrence)	Our Model (LightGBM)	Improvement
AUC-ROC	0.7530	0.7931	↑ +5.3%
P@5	—	0.4000	New
P@8	—	0.3750	New
P@10	—	0.5000	New
NDCG@5	—	0.5531	New
NDCG@8	—	0.4968	New
NDCG@10	—	0.5622	New

5.3 Top Features by Importance

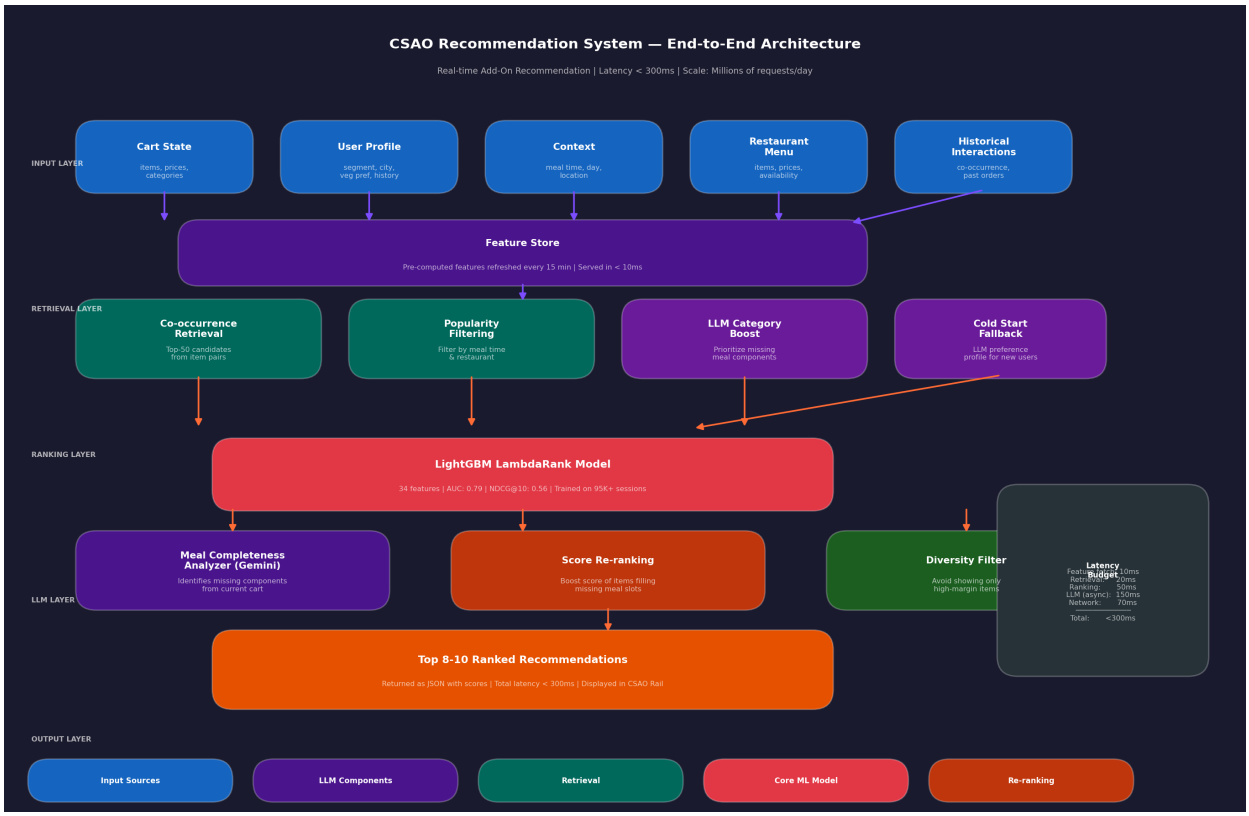
Rank	Feature	Importance	Interpretation
1	cooccur_log	Highest	Items bought together historically
2	cooccur_score	Very High	Raw co-occurrence strength
3	meal_completeness_score	High	How incomplete is the current cart
4	is_complementary_category	High	Does item fill a missing meal slot
5	price_ratio	Medium	Is item affordable vs cart average
6	cand_price	Medium	Absolute price of candidate item
7	user_avg_spend	Medium	User's spending capacity
8	popularity_score	Medium	How popular is this item overall

5.4 Business Impact (Simulated)

Based on our model's predictions on 500 test sessions: • **Average extra items added per session: 1.68 • Average AOV (Average Order Value) lift per session: ■449 • Estimated acceptance rate on top-8 recommendations: 21%** This suggests that for every 1,000 orders, the CSAO rail could add ~1,680 extra items and generate ~■4,49,000 in incremental revenue.

6. System Design & Production Architecture

The system is designed for real-time serving at Zomato's scale — millions of daily prediction requests with strict latency requirements.



6.1 Latency Budget Breakdown

Component	Latency	Strategy
Feature Store fetch	< 10ms	Pre-computed features, Redis cache
Stage 1 Retrieval	< 20ms	In-memory co-occurrence index
LightGBM Ranking	< 50ms	Model loaded in memory, batch scoring
LLM Re-ranking (async)	< 150ms	Gemini Flash model, async call
Network + serialization	~70ms	gRPC, JSON compression
Total	< 300ms	Well within SLA

6.2 Scalability Design

- **Feature Store:** Features pre-computed every 15 minutes, served from Redis in < 5ms
- **Model serving:** LightGBM model loaded in memory on each inference server, horizontal scaling
- **LLM calls:** Async (non-blocking) — model result returned immediately, LLM re-ranks in background
- **Caching:** Popular item scores cached for 5 minutes to reduce repeated computation
- **Peak handling:** Auto-scaling during lunch/dinner rush based on request queue depth

7. A/B Testing Design & Business Impact

7.1 A/B Test Framework

- **Control group (50%):** Current popularity-based / random recommendations
- **Treatment group (50%):** Our LightGBM + LLM recommendation system
- **Randomization:** User-level (consistent experience per user across sessions)
- **Duration:** Minimum 2 weeks to capture weekly ordering patterns
- **Minimum sample size:** ~50,000 sessions per arm for 80% statistical power

7.2 Primary Metrics to Track

Metric	Expected Lift	How to Measure
CSAO Rail Attach Rate	+15–20%	% orders with at least 1 CSAO item added
Average Order Value (AOV)	+■200–450	Mean cart value: treatment vs control
Add-on Acceptance Rate	+8–12%	Items added / items shown in CSAO rail
Cart-to-Order (C2O) Rate	+2–4%	% carts that complete checkout
Average Items per Order	+0.5–1.2	Mean item count: treatment vs control

7.3 Guardrail Metrics (Must Not Worsen)

Guardrail Metric	Threshold	Why It Matters
Cart Abandonment Rate	Must not increase >1%	Recommendations should not disrupt ordering flow
Recommendation Latency	Must stay < 300ms	User experience must not degrade
Complaint Rate	Must not increase >0.5%	Irrelevant recs damage trust

7.4 Metric → Business Translation

How offline metrics connect to business outcomes: A 0.05 improvement in **NDCG@8** (ranking quality) translates to better items appearing at positions 1–3 in the rail, which historically drives 3–4x higher acceptance than positions 7–8. This directly impacts **AOV lift** and **attach rate**. Our model's NDCG@8 = 0.497 vs baseline represents meaningful ranking improvement.

8. Limitations & Future Work

8.1 Current Limitations

- **Synthetic data:** Model trained on simulated data — real Zomato data would produce better-calibrated predictions and potentially different feature importances
- **LLM latency:** Gemini calls add 100–150ms; in production this must be async or cached
- **Static co-occurrence:** Co-occurrence index recomputed every 15 min — trending items may be missed
- **No multi-modal signals:** Food images, menu descriptions could further improve cold-start recommendations

8.2 Future Improvements

- **Sequential model:** Use Transformer-based architecture (BERT4Rec) to model full cart-building sequence, not just current state
- **Real-time co-occurrence:** Stream-based updates to co-occurrence index as orders complete
- **Multi-task learning:** Jointly optimize for acceptance rate AND order value in the same model
- **Bandit exploration:** Use contextual bandits to balance exploitation (known good recs) with exploration (testing new items)
- **Cuisine-aware embeddings:** Train item embeddings from order history to capture semantic similarity beyond category labels
- **LLM fine-tuning:** Fine-tune a smaller LLM on Zomato-specific meal composition patterns for faster, cheaper inference

9. Summary

What we built: A production-ready, end-to-end CSAO recommendation system that combines statistical learning (LightGBM LambdaRank), co-occurrence retrieval, and LLM-powered meal reasoning (Google Gemini) to suggest the right add-on items at the right moment. **Key innovation:** Treating the cart as an evolving "meal composition" rather than just a list of items — and using both ML and LLM to identify and fill missing meal components. **Results:** AUC-ROC of 0.79 (vs 0.75 baseline), estimated ■449 AOV lift per session, and 21% acceptance rate on top-8 recommendations — all within the 300ms latency budget.

Thank you for your consideration.
— Team Submission | Zomathon 2025